Derivation D4: Soundness and Completeness of the AVL Invariant Checker

Formal Verification of BST Order, Height Invariants, and Balance Factors

Simran (250001066) – Role P4 (Oracle & Verification Lead)
CS 253 / MA 253 Data Structures & Algorithms Lab

21 November 2026

1 Structural Invariants of an AVL Tree

An AVL tree over the ordered event domain $\mathcal{E}$ must satisfy three structural invariants simultaneously:

1. **BST Ordering Invariant:** For every node x, all nodes $y \in \text{LeftSubtree}(x)$ satisfy $y < x$, and all nodes $z \in \text{RightSubtree}(x)$ satisfy $x < z$ under the lexicographical order $(t(e), s(e))$.
2. **Stored Height Correctness:** For every node x, the cached attribute $x.h$ equals $1 + \max(h(x.\text{left}), h(x.\text{right}))$, with $h(\text{nullptr}) = -1$.
3. **Balance Factor Invariant:** For every node x, $\text{bf}(x) = h(x.\text{left}) - h(x.\text{right}) \in \{-1, 0, +1\}$.

2 The Validation Algorithm

The recursive function `validateInternal(Node* x, const Event* low, const Event* high)` verifies a subtree rooted at x within the open interval $(\text{low}, \text{high})$:

```
bool validateInternal(Node* x, const Event* low, const Event* high) {
    if (!x) return true;
    if (low && !( *low < x->event )) return false;
    if (high && !( x->event < *high )) return false;
    int b = ht(x->left) - ht(x->right);
    if (b < -1 || b > 1) return false;
    int expected_h = 1 + std::max(ht(x->left), ht(x->right));
    if (x->h != expected_h) return false;
    return validateInternal(x->left, low, &x->event) &&
           validateInternal(x->right, &x->event, high);
}
```

3 Soundness Proof

Theorem 1 (Soundness) *If `validateInternal(root, nullptr, nullptr)` returns `true`, then the tree rooted at `root` satisfies BST order, accurate stored heights, and the AVL balance condition at every node.*

Proof: By structural induction on the tree:

- **Base case:** When x = nullptr, the empty tree vacuously satisfies all three invariants, and the function returns `true`.
- **Inductive step:** Assume the theorem holds for both subtrees x.left and x.right.
  1. *BST ordering:* The parameter bounds enforce low $< x$.event $<$ high. By inductive hypothesis, all nodes in x.left fall in $(\text{low}, x.\text{event})$, so all left descendants are strictly less than x. Similarly, all right descendants fall in $(x.\text{event}, \text{high})$ and are strictly greater.
  2. *Stored height:* The check $x.h == 1 + \max(ht(x.\text{left}), ht(x.\text{right}))$ guarantees $x.h$ matches the actual subtree heights.
  3. *Balance factor:* The check $-1 \leq \text{bf}(x) \leq 1$ explicitly verifies the AVL invariant at node x.

  Since all conditions hold at x and in both subtrees, the entire tree satisfies all AVL invariants.

■

4 Completeness Proof

Theorem 2 (Completeness) *If any node in the tree violates BST ordering, stored height, or the balance factor constraint,* `validateInternal(root, nullptr, nullptr)` *returns* `false`.

Proof: Suppose an invariant violation exists at node u:

- **Case 1 (BST violation):** If u's key violates the binary search tree property relative to an ancestor, u will fall outside the valid $(\text{low}, \text{high})$ bound passed down from that ancestor, triggering an immediate `false`.
- **Case 2 (Height mismatch):** If $u.h \neq 1 + \max(ht(u.\text{left}), ht(u.\text{right}))$, the explicit equality check fails and returns `false`.
- **Case 3 (Balance factor violation):** If $|ht(u.\text{left}) - ht(u.\text{right})| > 1$, the balance check fails and returns `false`.

Because all returns are combined via logical `&&`, any `false` at node u propagates up to the root. ■

5 Complexity

The checker visits each of the n nodes exactly once, performing $O(1)$ operations per node. Runtime is $\Theta(n)$. The recursion stack depth equals tree height $h = O(\log n)$. Memory overhead is strictly $O(\log n)$ stack space with zero dynamic memory allocation.